Python Niveau I — Plan de cours détaillé

Formation : Initiation au langage Python **Durée** : 2 jours (14 heures) **Horaires** : 9h00 – 17h00

JOUR 1 — Découverte de Python

Module 1 : Prise en main et environnement (9h00 – 10h30)

Objectifs : Installer Python, comprendre les environnements de développement, exécuter ses premières commandes.

Contenu détaillé :

1. **Pourquoi Python ?** (15 min)
    - Python dans l'écosystème data science (comparaison R, Java, JavaScript)
    - Open source, communauté active, polyvalence
    - Cas d'usage : data science, web, automatisation, IA, sciences
2. **Installation de Python** (15 min)
    - Téléchargement depuis python.org
    - Vérification de l'installation : `python --version`
    - Le gestionnaire de paquets pip

3. **Les environnements de développement** (20 min)
    - VS Code : installation, extensions Python, terminal intégré
    - Jupyter Notebook : installation via pip, cellules de code et de texte
    - Autres options : PyCharm, IDLE (mention rapide)
4. **L'interpréteur interactif** (20 min)
    - Python comme calculatrice : opérations arithmétiques
    - Affectation avec `=`
    - Afficher avec `print()` et f-strings
    - L'historique des commandes
5. **Les scripts Python** (10 min)
    - Créer, sauvegarder, exécuter un script (.py)
    - Commentaires avec `#`
    - Exécuter depuis le terminal : `python script.py`
6. **Obtenir de l'aide** (10 min)
    - `help(fonction)`, `dir(objet)`
    - `type()` pour identifier un type
    - Documentation officielle docs.python.org

Activité : Créer un script qui effectue 10 calculs variés, les stocke dans des variables et les affiche avec des f-strings.

Module 2 : Types, texte et structures (10h45 – 12h15)

Objectifs : Identifier les types fondamentaux de Python, maîtriser les structures de données principales.

Contenu détaillé :

1. **Les types numériques** (10 min)
    - `int` (précision arbitraire), `float` (IEEE 754)
    - Opérateurs : `+`, `-`, `*`, `/`, `//`, `%`, `**`
    - Conversion : `int()`, `float()`
2. **Les chaînes de caractères** (15 min)
    - Guillemets simples, doubles, triples
    - f-strings : `f"Bonjour {nom}"`
    - Méthodes : `.upper()`, `.lower()`, `.strip()`, `.split()`, `.replace()`, `.join()`
    - Indexation et slicing sur les chaînes
    - Immutabilité des chaînes
3. **Booléens et None** (10 min)
    - `True`, `False`, opérateurs `and`, `or`, `not`
    - Valeurs « falsy » : `0`, `""`, `[]`, `{}`, `None`
    - `None` : absence de valeur
    - Comparaisons : `==`, `!=`, `<`, `>`, `<=`, `>=`
4. **Les listes** (15 min)
    - Création : `[]`, `list()`, `list(range())`
    - Mutabilité : modifier, ajouter, supprimer
    - Méthodes : `.append()`, `.extend()`, `.insert()`, `.remove()`, `.pop()`, `.sort()`, `.reverse()`
    - Opérations : `len()`, `in`, concaténation `+`, répétition `*`
5. **Indexation et slicing** (15 min)

    - Par position : `l[0]`, `l[-1]`
    - Slicing : `l[1:3]`, `l[:3]`, `l[2:]`, `l[::2]`, `l[::-1]`
    - Assignation par tranche : `l[1:3] = [10, 20]`
6. **Les tuples** (10 min)
    - Création : `()`, `tuple()`
    - Immutabilité : pas de modification après création
    - Unpacking : `a, b, c = (1, 2, 3)`
    - Quand utiliser un tuple vs une liste
7. **Les dictionnaires** (10 min)
    - Création : `{}`, `dict()`
    - Accès : `d["clé"]`, `d.get("clé", défaut)`
    - Méthodes : `.keys()`, `.values()`, `.items()`
    - Itérer sur un dictionnaire
8. **Les ensembles (sets)** (5 min)
    - Création : `set()`, `{1, 2, 3}`
    - Unicité des éléments
    - Opérations : union `|`, intersection `&`, différence `-`

Activité : Créer un mini-répertoire téléphonique (dictionnaire) avec recherche, ajout et suppression de contacts.

Module 3 : Contrôle, fonctions et logique (13h30 – 15h00)

Objectifs : Maîtriser le flux d'exécution et écrire ses premières fonctions.

Contenu détaillé :

1. **Conditions if/elif/else** (15 min)
    - Syntaxe avec indentation obligatoire
    - `if`, `elif`, `else`
    - Conditions multiples avec `and`, `or`
    - Opérateur ternaire : `x if condition else y`
2. **Boucle for** (15 min)
    - Itérer sur une liste, une chaîne, un dictionnaire
    - `range(start, stop, step)`
    - `enumerate()` pour index + valeur
    - `zip()` pour itérer en parallèle
    - `break` et `continue`
3. **Boucle while** (10 min)
    - Syntaxe et condition d'arrêt
    - Boucle infinie et `break`
    - Exemples : compteur, validation d'entrée
4. **Fonctions def** (20 min)
    - Syntaxe : `def nom(arg1, arg2):`
    - `return` explicite
    - Arguments par défaut : `def f(x, n=2):`
    - Arguments nommés : `f(x=5, n=3)`
    - Portée des variables (scope local vs global)
    - Docstrings : `"""Description de la fonction"""`
5. **Arguments avancés** (10 min)

    - `*args` : arguments positionnels variables
    - `**kwargs` : arguments nommés variables
    - Aperçu rapide
6. **Compréhensions de liste** (15 min)
    - Syntaxe : `[expr for x in iterable]`
    - Avec condition : `[expr for x in iterable if condition]`
    - Compréhensions de dictionnaire : `{k: v for k, v in ...}`
    - Comparaison avec boucle for classique
7. **Fonctions lambda** (5 min)
    - Syntaxe : `lambda x: x * 2`
    - Usage avec `sorted()`, `map()`, `filter()`
    - Quand utiliser lambda vs def

Activité : Écrire 3 fonctions — conversion Celsius/Fahrenheit, calcul d'IMC, nettoyage de texte — puis les appliquer à des listes avec des compréhensions.

Module 4 : Méthodologie de résolution Python (15h15 – 17h00)

Objectifs : Consolider les acquis des Modules 1 à 3 par la pratique guidée.

Contenu :

1. **Exercice guidé 1** (20 min) — Listes et slicing
    - Créer des listes de données, les filtrer, calculer des statistiques
2. **Exercice guidé 2** (20 min) — Dictionnaires
    - Construire un dictionnaire de produits, calculer des totaux
3. **Exercice guidé 3** (20 min) — Fonctions

    - Écrire une fonction de validation d'email
    - L'appliquer à une liste d'adresses
4. **Exercice guidé 4** (25 min) — Mini-analyse
    - Créer un jeu de données (liste de dictionnaires), calculer des statistiques par groupe
5. **Correction collective et questions** (20 min)

JOUR 2 — Manipulation de données

Module 5 : Fichiers, modules et erreurs (9h00 – 10h30)

Objectifs : Lire et écrire des fichiers, utiliser les modules standards et externes.

Contenu détaillé :

1. **Lire un fichier texte** (15 min)
    - `open("fichier.txt", "r")` : modes r, w, a
    - `.read()`, `.readline()`, `.readlines()`
    - Itérer ligne par ligne
2. **Le with statement** (10 min)
    - Fermeture automatique du fichier
    - `with open("f.txt") as f:`
    - Bonnes pratiques de gestion de ressources
3. **Écrire dans un fichier** (10 min)
    - Mode `"w"` (écraser) vs `"a"` (ajouter)
    - `.write()`, `.writelines()`

    - Encodage UTF-8 : `encoding="utf-8"`

4. **Le module csv** (15 min)
    - `csv.reader()` et `csv.writer()`
    - `csv.DictReader()` et `csv.DictWriter()`
    - Paramètres : `delimiter`, `quotechar`
5. **Le module json** (10 min)
    - `json.load()` / `json.dump()` : fichiers
    - `json.loads()` / `json.dumps()` : chaînes
    - `indent` pour un affichage lisible
6. **os et chemins de fichiers** (10 min)
    - `os.path.join()`, `os.path.exists()`
    - `os.listdir()`, `os.getcwd()`
    - `pathlib.Path` (aperçu moderne)
7. **Importer des modules** (15 min)
    - `import math`, `from math import sqrt`
    - `import module as alias`
    - Modules standards utiles : `math`, `datetime`, `collections`, `random`
    - `pip install package` pour les modules externes
8. **Créer son propre module** (5 min)
    - Un fichier .py = un module
    - `if __name__ == "__main__":`

Activité : Lire un CSV contenant des données de ventes, le traiter en Python pur, et écrire un rapport JSON.

Module 6 : Structuration du code et validation (10h45 – 12h15)

Objectifs : Organiser un script Python en mini-projet maintenable et fiable.

Contenu détaillé :

1. **Découper un problème** (15 min)
    - Identifier entrées, traitement, sorties
    - Pseudo-code avant implémentation
    - Responsabilité unique par fonction
2. **Structurer en fonctions** (20 min)
    - Fonctions courtes et lisibles
    - Paramètres explicites et valeurs de retour stables
    - Éviter la duplication de logique
3. **Validation des données** (15 min)
    - Contrôler types, plages de valeurs, champs obligatoires
    - Lever des exceptions explicites (`ValueError`)
4. **Gestion d'erreurs** (15 min)
    - `try/except` ciblés
    - Messages d'erreur compréhensibles
    - Différence erreur métier vs erreur technique
5. **Organisation des modules** (15 min)
    - Un fichier = un module cohérent
    - Constantes de configuration séparées
    - Point d'entrée avec `if __name__ == "__main__":`
6. **Contrôles qualité de base** (10 min)

- Cas nominal, cas limite, cas erreur
- Checklist avant livraison d'un script

Activité : Refactor d'un script monolithique en fonctions/modules avec validation et gestion d'erreurs.

Module 7 : Visualisation matplotlib (Python pur) (13h30 – 15h00)

Objectifs : Créer des graphiques informatifs avec matplotlib.

Contenu détaillé :

1. **Les bases de matplotlib** (10 min)
   - `import matplotlib.pyplot as plt`
   - `plt.figure()`, `plt.show()`, `plt.close()`
   - Interface pyplot vs interface orientée objet (`fig, ax = plt.subplots()`)
2. **Nuage de points (scatter)** (10 min)
   - `plt.scatter(x, y)` ou `ax.scatter(x, y)`
   - Paramètres : `c` (couleur), `s` (taille), `alpha` (transparence), `marker`
   - Lecture de tendance simple avec une courbe de référence
3. **Courbes (line plot)** (10 min)
   - `plt.plot(x, y)` : style, couleur, marqueurs
   - Plusieurs courbes sur le même axe
   - `plt.fill_between()` pour les zones
4. **Histogramme** (10 min)
   - `plt.hist(data, bins=15)`
   - `color`, `edgecolor`, `alpha`

- Ajouter moyenne et médiane avec `plt.axvline()`
- `density=True` pour les densités

5. **Boîte à moustaches (boxplot)** (10 min)
    - `plt.boxplot(data)`
    - Boxplot par groupe
    - Interprétation : médiane, quartiles, outliers
    - `vert=False` pour l'orientation horizontale
6. **Diagramme en barres (bar)** (10 min)
    - `plt.bar(x, height)` et `plt.barh()`
    - Barres empilées et côte à côte
    - Ajouter les valeurs au-dessus des barres
7. **Multi-graphiques (subplots)** (10 min)
    - `fig, axes = plt.subplots(2, 2, figsize=(10, 8))`
    - Accéder aux axes : `axes[0, 0]`, `axes[1, 1]`
    - `plt.tight_layout()` pour l'espacement
8. **Personnalisation et export** (10 min)
    - Titres : `plt.title()`, `plt.xlabel()`, `plt.ylabel()`
    - Légende : `plt.legend()`
    - Style : `plt.style.use("ggplot")`
    - Export : `plt.savefig("figure.png", dpi=150, bbox_inches="tight")`

Activité : Créer un dashboard de 4 graphiques à partir de listes Python issues d'un CSV.

Module 8 : Projet de synthèse Python pur (15h15 – 17h00)

Objectifs : Mobiliser l'ensemble des compétences acquises sur un cas concret.

Contenu :

1. **Présentation du projet** (10 min)
    - Jeu de données : tips (pourboires dans un restaurant)
    - Objectif : analyser les facteurs influençant le montant des pourboires
2. **Étape 1 : Import et exploration** (20 min)
    - Charger les données avec `csv.DictReader`
    - Examiner la structure, les types, les valeurs manquantes
    - Statistiques descriptives globales
3. **Étape 2 : Nettoyage** (15 min)
    - Traitement des valeurs manquantes
    - Création de variables dérivées (pourcentage de pourboire, catégories)
4. **Étape 3 : Statistiques descriptives** (15 min)
    - Moyennes par groupe et médianes
    - Agrégations avec dictionnaires Python
5. **Étape 4 : Visualisations** (20 min)
    - 4 graphiques commentés (scatter, hist, boxplot, barplot)
    - Dashboard multi-graphiques
6. **Étape 5 : Synthèse et interprétation** (10 min)
    - Rédiger 5 conclusions en commentaires Python
7. **Correction collective et bilan** (15 min)
    - Tour de table, questions, ressources pour aller plus loin

---

Repères de code du niveau I

```python
def moyenne(notes):
    return sum(notes) / len(notes)

notes = [12, 15, 9, 14]
print(moyenne(notes))

import csv

with open("donnees.csv", "r", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

totaux = {}
for r in rows:
    cat = r["categorie"]
    totaux[cat] = totaux.get(cat, 0.0) + float(r["montant"])
print(totaux)
```

Notions théoriques transverses (niveau I)

- **Mutabilité et pièges** : effets de bord lors du passage d'une liste à une fonction.
- **Références objet** : comprendre pourquoi `b = a` ne crée pas de copie.
- **Design de fonctions** : une responsabilité claire, arguments explicites, valeur de retour stable.
- **Lecture d'erreurs** : décoder traceback, type d'exception et ligne fautive.
- **Qualité de base** : nommage, indentation, commentaires utiles, tests manuels systématiques.

```python
def add_item(items: list[str], value: str) -> list[str]:
    # évite de modifier la liste d'origine
    new_items = items.copy()
```

```
    new_items.append(value)

    return new_items
```

Définitions techniques complémentaires (niveau I)

- **Runtime Python** : environnement d'exécution qui gère objets, exceptions et appels de fonctions.
- **Binding** : association d'un nom Python à une référence d'objet en mémoire.
- **Objet mutable** : objet modifiable en place (ex: liste, dictionnaire).
- **Objet immuable** : objet non modifiable après création (ex: tuple, chaîne, entier).
- **Slicing** : extraction d'une sous-séquence par indices `[start:stop:step]`.
- **Idempotence** : propriété d'une opération qui peut être rejouée sans dérive de résultat.
- **Exception** : signal d'erreur interrompant le flux normal d'exécution.
- **Traceback** : pile d'appels détaillant où et pourquoi une erreur s'est produite.
- **Contrat de fonction** : attentes sur les entrées, sorties et erreurs possibles.
- **Encodage** : règle de transformation texte↔octets (UTF-8 recommandé).

Détail technique par domaine (niveau I)

Mémoire et références

- Expliquer la différence entre égalité de valeur (`==`) et identité d'objet (`is`).
- Montrer les effets de bord d'un passage d'objet mutable à une fonction.
- Utiliser une copie défensive (`copy()`) quand l'appelant doit conserver son état.

Complexité des opérations courantes

- Rappeler que l'accès indexé d'une liste est en moyenne O(1), alors que la

recherche est O(n).

- Insister sur l'intérêt des dictionnaires pour les accès par clé en moyenne O(1).
- Préciser que le tri (`sorted`) est en O(n log n) et doit être évité dans des boucles inutiles.

Validation et gestion d'erreurs

- Valider les entrées en début de fonction pour échouer tôt avec un message explicite.
- Capturer uniquement les exceptions attendues, éviter les `except Exception` génériques.
- Chaîner les exceptions avec `raise ... from err` pour préserver le diagnostic.

Fichiers, formats et encodage

- Ouvrir les fichiers texte avec `encoding="utf-8"` pour garantir la portabilité.
- Séparer la lecture brute, la validation du schéma et la transformation métier.
- Tracer les lignes rejetées (motif, valeur) pour faciliter la correction des sources.

```
def parse_amount(raw: str) -> float:
    if raw is None or raw.strip() == "":
        raise ValueError("montant manquant")
    try:
        value = float(raw)
    except ValueError as err:
        raise ValueError(f"montant invalide: {raw!r}") from err
    if value < 0:
        raise ValueError("montant négatif interdit")
    return value
```